ADVANCED SQL TRAINING

Corporate Training Series — training_db

100 Practice Questions

Select · Joins · Subqueries · Correlated Subqueries

Basic to Intermediate Level | Answer key provided in separate document

E-Commerce Schema | MySQL 8+ | 100 Questions

How to Use This Workbook

All queries below operate on the training_db database (E-Commerce schema). Set up the database using training_db_setup.sql before attempting the exercises.

Basic	Core SELECT syntax, filtering, ORDER BY, simple aggregates, basic JOINs
Intermediate	Multi-table JOINs, GROUP BY + HAVING, subqueries, correlated subqueries, views, indexes, DCL
Tip	Try writing each query yourself before looking at the answer document
Schema	Tables: customers, products, orders, order_items, employees, accounts

Section 1 — SELECT, WHERE & Operators

Q1. **[Basic]** *SELECT – All Columns*

Write a query to display all columns and all rows from the `customers` table.

Q2. **[Basic]** *SELECT – Specific Columns*

Write a query to display only the `name` and `city` columns from the `customers` table.

Q3. **[Basic]** *WHERE – Equality*

List all customers who live in 'Mumbai'.

Q4. **[Basic]** *WHERE – Inequality*

List all products that are NOT in the 'Electronics' category.

Q5. **[Basic]** *WHERE – Numeric Comparison*
Find all products whose price is greater than ₹10,000.

Q6. **[Basic]** *ORDER BY – ASC*
Display all products sorted by price from lowest to highest.

Q7. **[Basic]** *ORDER BY – DESC*
Display all orders sorted by order_date from newest to oldest.

Q8. **[Basic]** *DISTINCT*
List all unique cities from which customers have registered.

Q9. **[Basic]** *LIMIT*

Show the top 3 most expensive products.

Q10. **[Basic]** *BETWEEN*
Find all products priced between ₹2,000 and ₹15,000 (inclusive).

Q11. **[Basic]** *IN*
List all customers who live in Mumbai, Delhi, or Bangalore.

Q12. **[Basic]** *NOT IN*
Find all orders whose status is neither 'cancelled' nor 'pending'.

Q13. **[Basic]** *LIKE – Prefix*
Find all customers whose name starts with the letter 'A'.

Q14. **[Basic]** *LIKE – Contains*
Find all products whose name contains the word 'Chair'.

Q15. **[Basic]** *LIKE – Single Character*
Find all customers whose name has exactly 8 characters.

Q16. **[Basic]** *IS NULL*
Find all employees who do NOT have a manager (i.e., manager_id is NULL).

Q17. **[Basic]** *IS NOT NULL*
Find all employees who DO have a manager.

Q18. **[Basic]** *AND*
Find all Electronics products with stock greater than 50.

Q19. **[Basic]** *OR*

Find all orders that are either 'delivered' or have a total greater than ₹20,000.

Q20. **[Basic]** *NOT*

List all products that are not in the 'Furniture' category and are priced under ₹5,000.

Section 2 — Aggregate Functions & GROUP BY

Q21. **[Basic]** *COUNT(*)*
Count the total number of customers in the database.

Q22. **[Basic]** *COUNT with WHERE*
Count how many orders have the status 'delivered'.

Q23. **[Basic]** *SUM*
Find the total revenue from all orders combined.

Q24. **[Basic]** *AVG*
Find the average price of all products.

Q25. **[Basic]** *MIN / MAX*
Find the cheapest and most expensive product prices.

Q26. **[Intermediate]** *GROUP BY*
Count the number of products in each category.

Q27. **[Intermediate]** *GROUP BY + SUM*
Find the total stock available for each product category.

Q28. **[Intermediate]** *GROUP BY + AVG*
Find the average order total for each order status.

Q29. **[Intermediate]** *HAVING*
List product categories that have more than 1 product.

Q30. **[Intermediate]** *WHERE + GROUP BY + HAVING*
Among orders placed in 2023, find customers who have placed more than 1 order.

Section 3 — INNER JOIN

Q31. **[Basic]** *INNER JOIN – 2 Tables*
List all orders along with the name of the customer who placed each order.

Q32. **[Basic]** *INNER JOIN – Filter*
Find all delivered orders and display the customer's name, city, and order total.

Q33. **[Intermediate]** *INNER JOIN – 3 Tables*
List all order line items showing customer name, product name, quantity, and unit price.

Q34. **[Intermediate]** *INNER JOIN – 4 Tables + Aggregate*
For each customer, calculate the total quantity of items ordered across all their orders.

Q35. **[Intermediate]** *INNER JOIN + GROUP BY*
Find the total revenue generated by each product (qty × unit_price summed).

Section 4 — LEFT JOIN

Q36. **[Basic]** *LEFT JOIN – All Rows*

List all customers and their orders. Also show customers who have placed no orders (show NULL for order columns).

Q37. **[Basic]** *LEFT JOIN – Find Unmatched*

Find all customers who have NEVER placed any order.

Q38. **[Intermediate]** *LEFT JOIN – Products Not Ordered*

Find all products that have never been included in any order.

Q39. **[Intermediate]** *LEFT JOIN + COUNT*

For each customer, show how many orders they have placed (include customers with 0 orders).

Q40. **[Intermediate]** *LEFT JOIN + COALESCE*

List all customers with their total spending. Show 0 for customers who haven't spent anything (use COALESCE).

Section 5 — RIGHT JOIN & FULL OUTER JOIN

Q41. **[Intermediate]** *RIGHT JOIN*

List all orders and the customer details for each. Include orders where the customer record may be missing.

Q42. **[Intermediate]** *FULL OUTER JOIN (UNION trick)*

Write a query that shows ALL customers and ALL orders — highlighting unmatched rows on either side.

Section 6 — SELF JOIN

Q43. **[Intermediate]** *SELF JOIN – Hierarchy*

List each employee along with their manager's name. For top-level employees (no manager), show NULL.

Q44. **[Intermediate]** *SELF JOIN – Salary Comparison*

Find all employees who earn MORE than their direct manager.

Q45. **[Intermediate]** *SELF JOIN – Same Department*

List all pairs of employees who work in the same department (no duplicates, no self-pairing).

Section 7 — Scalar & IN Subqueries

Q46. **[Intermediate]** *Scalar Subquery – WHERE*
Find all products priced above the average product price.

Q47. **[Intermediate]** *Scalar Subquery – SELECT*
Show each product's name, price, and the difference between its price and the overall average price.

Q48. **[Intermediate]** *Scalar Subquery – HAVING*
Find all order statuses where the average order total exceeds the overall average order total.

Q49. **[Intermediate]** *Scalar Subquery – Most Recent*
Retrieve the details of the single most recent order placed.

Q50. **[Intermediate]** *Scalar Subquery – MIN Salary*
Find the employee(s) who earn the minimum salary in the company.

Q51. **[Intermediate]** *IN Subquery*
Find all customers who have placed at least one order.

Q52. **[Intermediate]** *NOT IN Subquery*
Find all products that appear in NO order_items rows (unsold inventory) using NOT IN.

Q53. **[Intermediate]** *IN Subquery – Nested 2 Levels*
Find all customers who ordered at least one Electronics product.

Section 8 — EXISTS & Derived Table

Q54. **[Intermediate]** *EXISTS*
List all customers who have placed at least one order — using EXISTS.

Q55. **[Intermediate]** *NOT EXISTS*
List customers who have NEVER placed any order — using NOT EXISTS.

Q56. **[Intermediate]** *EXISTS – Conditional*
Find all products for which there is at least one order item with qty >= 2.

Q57. **[Intermediate]** *Derived Table*
Using a derived table, find the top 3 customers by total spend.

Q58. **[Intermediate]** *Derived Table – Category Revenue*
Using a derived table, find all product categories whose total revenue (qty × unit_price) exceeds ₹50,000.

Q59. **[Intermediate]** *Derived Table – Average of Aggregates*

Find the average number of items per order across all orders (use a derived table to first count per order).

Section 9 — Correlated Subqueries [Focus]

Q60. **[Intermediate]** *Correlated Subquery – Category Avg*
Find all products whose price is above the average price of their OWN category.

Q61. **[Intermediate]** *Correlated Subquery – Highest Per Group*
Find the most expensive product in EACH category using a correlated subquery.

Q62. **[Intermediate]** *Correlated Subquery – Employee Dept Avg*
List employees whose salary is above the average salary in their department.

Q63. **[Intermediate]** *Correlated Subquery – Customer Last Order*
For each customer, display their most recent order date using a correlated subquery in SELECT.

Q64. **[Intermediate]** *Correlated Subquery – Nth Salary*
Using a correlated subquery, find the second highest salary in the employees table.

Q65. **[Intermediate]** *Correlated Subquery – Row Rank*

List each order with its rank by total (highest = 1) within the same customer, using a correlated subquery.

Section 10 — Set Operations

Q66. **[Intermediate]** *UNION*

Produce a single list of all city names from customers and all dept names from employees, with no duplicates.

Q67. **[Intermediate]** *UNION ALL*

Show a combined list of order IDs from orders with status 'pending' and orders with total > ₹10,000 (include duplicates).

Q68. **[Intermediate]** *INTERSECT (using JOIN)*

Find customer_ids that appear in BOTH the customers table and as a customer_id in the orders table (INTERSECT pattern using JOIN).

Q69. **[Intermediate]** *EXCEPT/MINUS (using NOT IN)*

Find all customer_ids in the customers table that do NOT appear in the orders table (EXCEPT pattern).

Section 11 — Aliases & Expressions

Q70. **[Basic]** *Column Alias – Calculated*
Display each product's name, price, and price with 18% GST added. Label the GST column as `price_with_gst`.

Q71. **[Basic]** *Table Alias*
Rewrite the query: 'List all orders with the customer name' — using single-letter table aliases.

Q72. **[Intermediate]** *Alias in ORDER BY*
Compute each employee's annual salary (monthly × 12) and sort by it descending. Alias as `annual_salary`.

Section 12 — Views & Indexes

Q73. **[Intermediate]** *CREATE VIEW*

Create a view named `vw_customer_orders` that shows customer name, city, order_id, order_date, and total for all orders.

Q74. **[Intermediate]** *VIEW + WHERE*

Query the view `vw_customer_orders` to show only orders with total > ₹10,000.

Q75. **[Intermediate]** *CREATE OR REPLACE VIEW*

Modify the view `vw_customer_orders` to also include the order status column.

Q76. **[Intermediate]** *CREATE INDEX*

Create a non-clustered index on the `status` column of the `orders` table.

Q77. **[Intermediate]** *Composite Index*

Create a composite index on `orders(customer_id, order_date)` to speed up queries that filter by customer and sort by date.

Q78. **[Intermediate]** *UNIQUE INDEX*

Create a unique index on the `email` column of the `customers` table to prevent duplicates.

Section 13 — DCL: GRANT & REVOKE

Q79. **[Intermediate]** *CREATE USER + GRANT*

Create a new database user `sales_rep` on localhost and grant SELECT access on all tables in `training_db`.

Q80. **[Intermediate]** *Column-Level GRANT*

Grant the user `intern1` SELECT access to ONLY the name and city columns of the `customers` table.

Q81. **[Intermediate]** *REVOKE*

Revoke the SELECT privilege on `training_db.orders` from the user `sales_rep`.

Q82. **[Intermediate]** *GRANT via ROLE*

Create a role `analyst_role`, grant it SELECT on all tables in training_db, then assign the role to user `alice`.

Section 14 — LIMIT & OFFSET

Q83. **[Basic]** *LIMIT*

Show the 5 most recently joined customers.

Q84. **[Intermediate]** *LIMIT + OFFSET (Pagination)*

Implement pagination for the products table: show page 2 with 2 products per page, ordered by price ascending.

Q85. **[Intermediate]** *LIMIT with Subquery*

Find the 2nd and 3rd most expensive products using LIMIT and OFFSET.

Section 15 — Mixed & Comprehensive Queries

Q86. **[Intermediate]** *JOIN + Subquery*
Find all customers whose latest order total is greater than ₹10,000.

Q87. **[Intermediate]** *JOIN + GROUP BY + HAVING*
Find all customers who have spent more than ₹20,000 in total across all their orders.

Q88. **[Intermediate]** *Correlated + EXISTS*
List all products that have been ordered more than once (appear in multiple order_item rows).

Q89. **[Intermediate]** *SELF JOIN + Correlated*
For each employee, show their salary percentile within their department (what fraction of colleagues they out-earn, as a percentage).

Q90. **[Intermediate]** *Derived Table + JOIN*
Find the customer who placed the highest single-order value. Show their name, city, and that order total.

Q91. **[Intermediate]** *Multiple Aggregates + HAVING*

Find products that have been ordered at least 3 times in total (across all order_items rows) AND have total quantity sold > 5.

Q92. **[Intermediate]** *UNION + Aggregate*

Show a single result set of: (a) most expensive product name & price, and (b) least expensive product name & price.

Q93. **[Intermediate]** *LIKE + JOIN*

Find all orders placed by customers whose names start with 'A', along with the total amount.

Q94. **[Intermediate]** *EXISTS + JOIN*

Find all orders that contain at least one product from the 'Electronics' category — using EXISTS.

Q95. **[Intermediate]** *NOT EXISTS + Correlated*

Find customers who have never ordered any product from the 'Furniture' category.

Q96. **[Intermediate]** *View + Subquery*

Create a view `vw_high_value_customers` that lists customers whose total spending (across all orders) is above the average spending of all customers.

Q97. **[Intermediate]** *Correlated – Update Scenario*

Write a SELECT that shows what each product's stock would look like after reducing it by the total quantity sold. (Read-only: no UPDATE needed.)

Q98. **[Intermediate]** *3-Level Nested Subquery*

Find the name of the customer who placed the order with the single highest total value, using nested subqueries only (no JOIN).

Q99. **[Intermediate]** *Full Scenario – Sales Report*

Generate a complete sales report: for each customer show their name, city, number of orders, total spend, average order value, and their highest single order — for customers who have spent more than ₹10,000 in total.

Q100. **[Intermediate]** *Comprehensive – Inventory + Sales*

For each product, show: name, category, current stock, total units sold, projected remaining stock, and revenue generated. Sort by revenue descending. Include products with no sales (show 0 for sold/revenue).